

VIT BHOPAL UNIVERSITY

Vellore Institute of Technology, Bhopal

Project Report on

Student Grade Manager

A Python Console Application for Managing Student Marks

Submitted by	Hrugved Sharad Rathod
Registration No.	26BCE11362
Course	CSE1021
Faculty Mentor	Dr. Poonkuntran S.
Date of Submission	29 September 2026

Table of Contents

1. Introduction	3
2. Problem Statement	3
3. Functional Requirements	3
4. Non-functional Requirements	3
5. System Architecture	4
6. Design Diagrams	5
6.1 Use Case Diagram	5
6.2 Workflow Diagram	6
6.3 Sequence Diagram	7
6.4 Class / Component Diagram	8
6.5 ER Diagram	9
7. Design Decisions & Rationale	9
8. Implementation Details	9
8.1 Data storage and averaging	10
8.2 Adding a student with validation	10
8.3 Finding the topper	10
8.4 The main loop	10
9. Results	11
10. Testing Approach	12
11. Challenges Faced	13
12. Learnings & Key Takeaways	14
13. Future Enhancements	14
14. References	14

1. Introduction

The Student Grade Manager is a menu-driven console application written in Python 3. It lets a teacher or student record the marks of a group of students, view every student with a computed average, identify the top and lowest performers, correct marks that were entered wrongly, and remove students who are no longer needed.

The program is intentionally small and dependency-free: it uses only Python built-in types (a dictionary and lists), built-in functions and the standard `input()` and `print()` functions. That keeps the focus on the fundamentals taught in the course: decomposing a problem into functions, controlling flow with loops and conditions, validating user input, and handling errors with `try/except`.

2. Problem Statement

Keeping marks on paper or in an unstructured list makes it slow to compute averages, spot the best and weakest performers, or fix mistakes. The goal of this project is to build a simple, reliable console tool that stores each student's marks under their name and provides the common grade-book operations through a numbered menu, while rejecting invalid input politely instead of crashing.

3. Functional Requirements

ID	Requirement
FR1	Add a student with a unique, non-empty name and one or more integer marks entered on one line, separated by spaces.
FR2	Reject duplicate names and direct the user to the update option.
FR3	View all students, showing each name, marks list and average to two decimal places.
FR4	Find and display the topper (highest average) and the lowest scorer (lowest average).
FR5	Update the marks of an existing student, replacing the old marks list.
FR6	Remove an existing student from the system.
FR7	Show a numbered main menu repeatedly until the user chooses Exit.
FR8	Display a clear message for invalid menu choices, empty input, non-numeric marks, unknown names and an empty student list.

4. Non-functional Requirements

- **Usability:** plain-language prompts, a numbered menu, and a blank line after every result for readability.
- **Reliability:** no input the user can type should crash the program; all errors produce a message and return to the menu.

- **Performance:** every operation is a single pass over the students ($O(n)$), which is instantaneous for class-sized data.
- **Maintainability:** one function per operation, small helper functions, and consistent naming make the code easy to read and extend.
- **Portability:** runs on any system with Python 3 and needs no third-party libraries.
- **Simplicity:** data is held in memory only, so there is nothing to install or configure (persistence is listed under future enhancements).

5. System Architecture

The application follows a simple layered structure inside a single source file, `student_grade_manager.py`:

- **Presentation layer:** `show_menu()` and the `main()` loop print the menu, read the choice and dispatch it.
- **Operation layer:** six functions (`add_student`, `view_all_students`, `find_topper`, `find_lowest_scorer`, `update_marks`, `remove_student`), each implementing one use case, including its own input validation.
- **Logic layer:** `calculate_average(marks)` is the shared helper used by the view, topper and lowest-scorer operations.
- **Data layer:** a module-level dictionary `students` that maps each name (string) to a list of integer marks.

Item	Choice
Language	Python 3
Interface	Command-line (console), menu driven
Data structure	dict of str to list[int], held in memory
External libraries	None
Entry point	<code>main()</code> called at the end of the file

6. Design Diagrams

6.1 Use Case Diagram

A single actor, the teacher or user, interacts with the system through seven use cases that correspond one-to-one with the menu options.

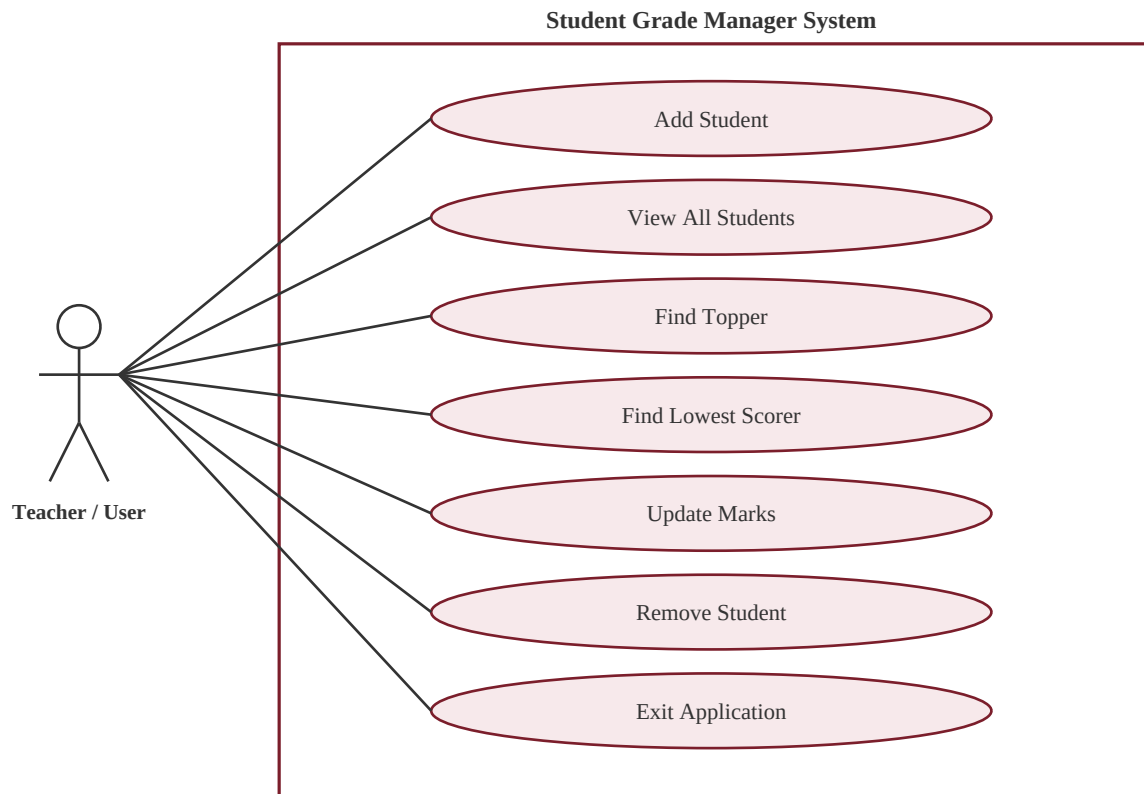


Figure 1: Use case diagram

6.2 Workflow Diagram

The workflow shows the main loop: the menu is displayed, a choice is read and validated, the matching function runs, and control returns to the menu until Exit is selected.

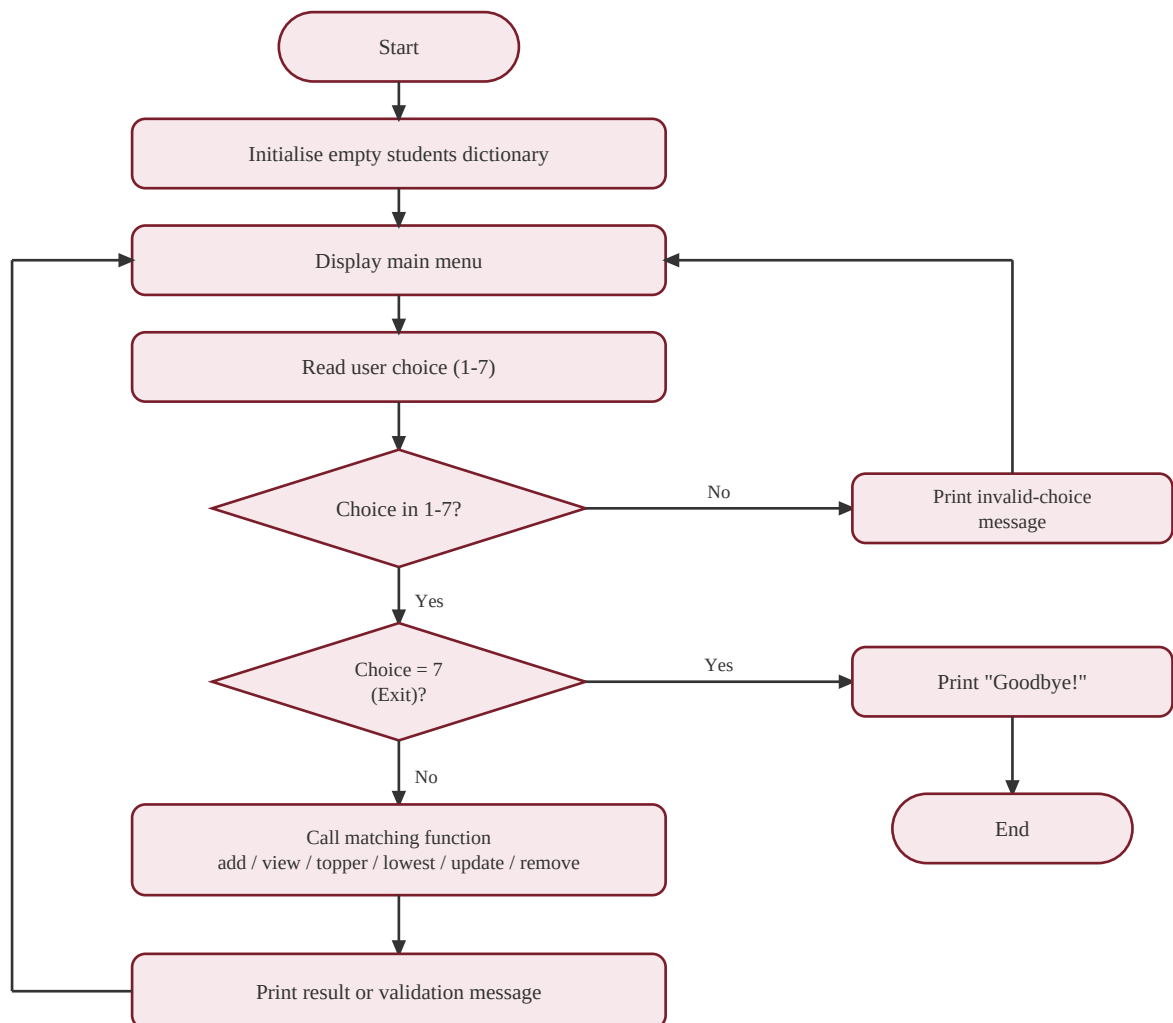


Figure 2: Workflow (flowchart) of the main menu loop

6.3 Sequence Diagram

The sequence below traces the *Add Student* use case for a new, valid student, showing the prompts, the duplicate check against the dictionary, validation of the marks and the final store operation.

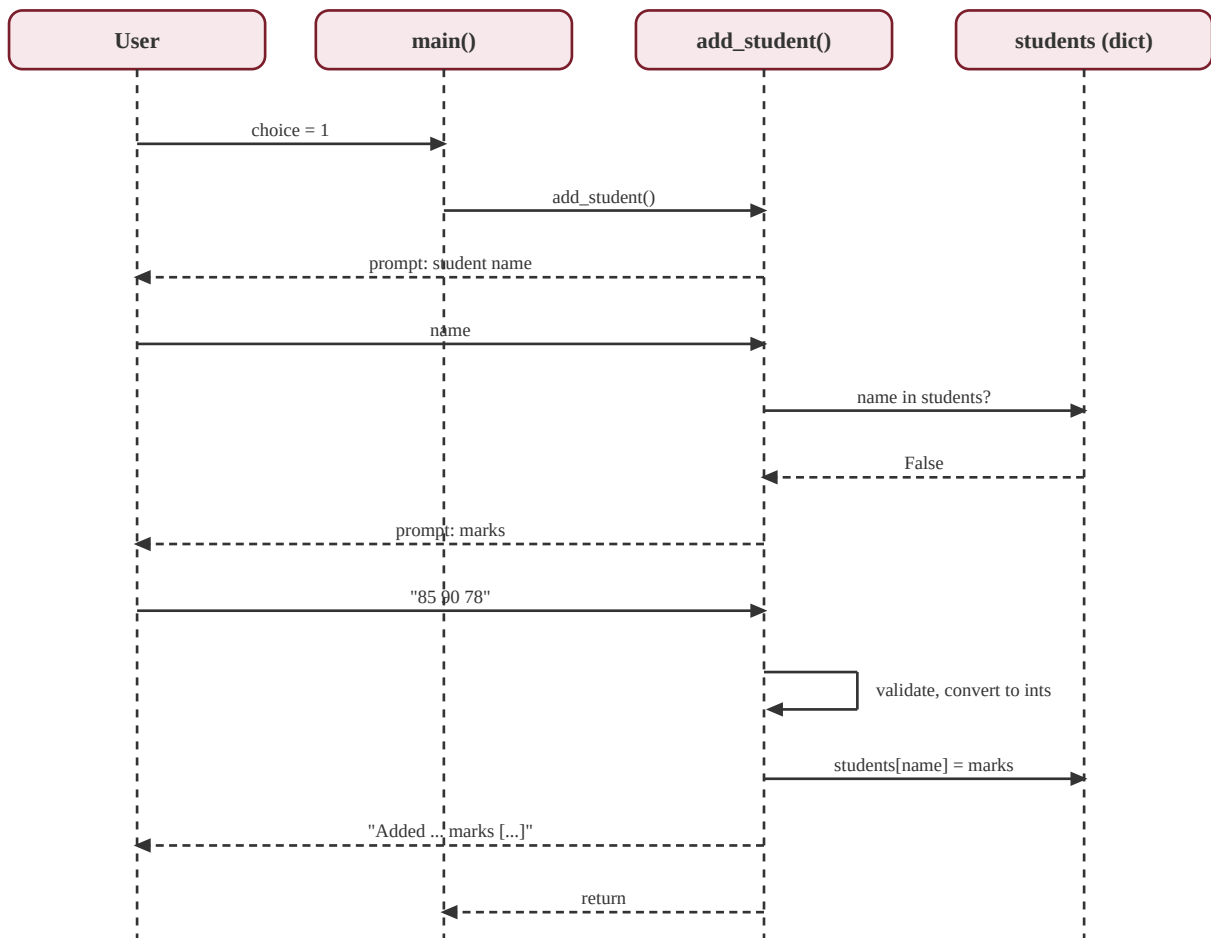
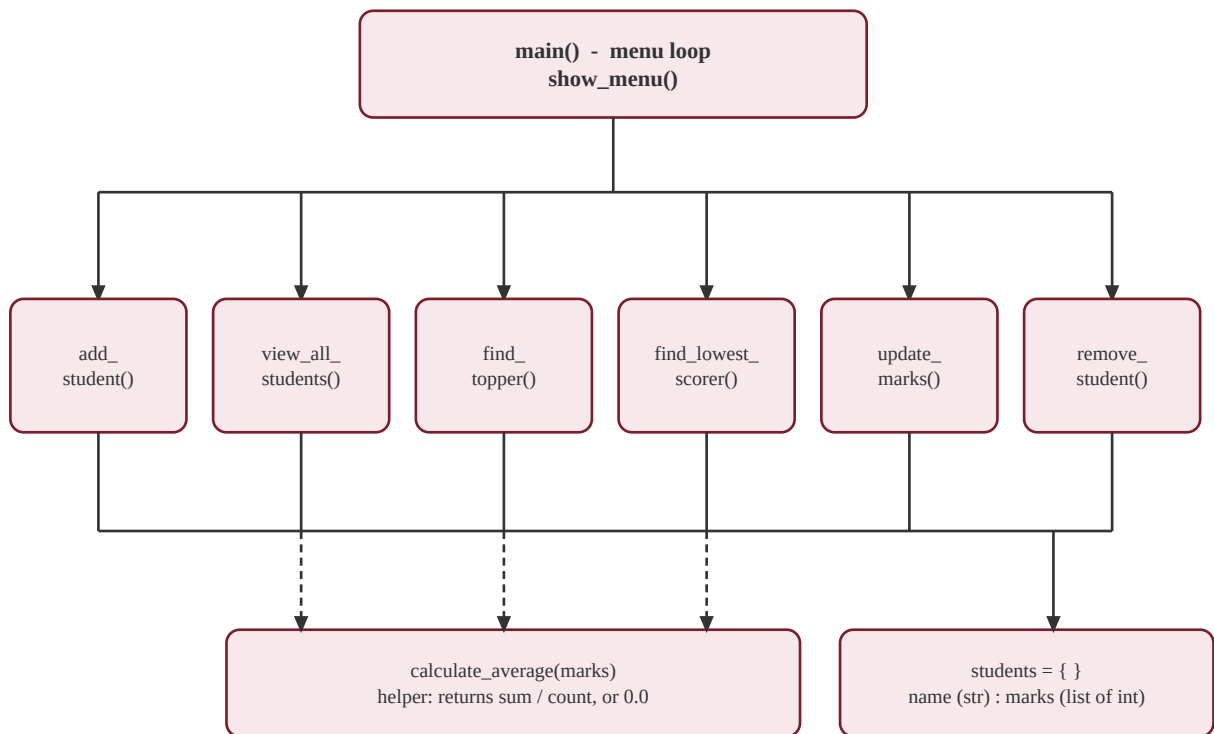


Figure 3: Sequence diagram for adding a student

6.4 Class / Component Diagram

The program is procedural and defines no classes, so its structure is shown as a component diagram of functions and the shared data they use.

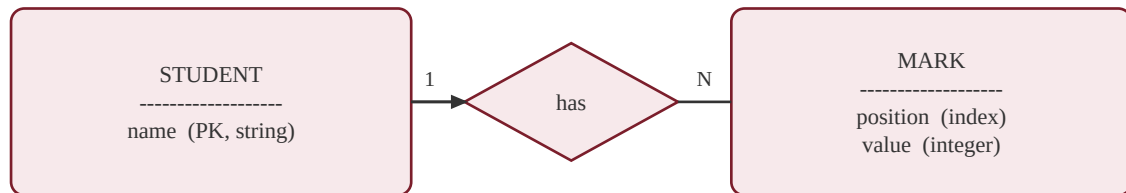


solid arrow = reads / writes the students dictionary dashed arrow = calls calculate_average()
(Python functions grouped as components; the program uses no classes)

Figure 4: Component diagram of functions and data

6.5 ER Diagram

Because the data lives only in memory there is no physical database, so an ER diagram is not strictly required. The logical relationship is still shown for clarity and for planning a future database version: one student has many marks.



Logical model only: the program keeps this data in memory as `students[name] = [mark1, mark2, ...]` and has no database or file storage.

Figure 5: Logical entity-relationship model

7. Design Decisions & Rationale

Decision	Rationale
Dictionary keyed by student name	Gives $O(1)$ lookup for the duplicate, update and remove checks and keeps the code short.
Marks stored as a list of integers	Allows any number of marks per student and makes sum/len averaging trivial.
Average calculated on demand, not stored	Prevents the stored average from ever being out of date after an update.
One function per menu option	Each use case can be read, tested and changed independently; the main loop stays a simple dispatcher.
<code>calculate_average</code> helper with an empty-list guard	Removes duplicated logic and avoids division by zero.
Validation with <code>strip()</code> , empty checks and <code>try/except ValueError</code>	Stops stray spaces, blank input and non-numeric marks from crashing the program.
Early <code>return</code> after each error	Keeps functions flat and avoids deeply nested if statements.
Strict <code>></code> and <code><</code> comparisons for topper and lowest	On equal averages the student added first is reported, giving a predictable result.
Console interface	Matches the scope of an introductory course and needs no extra tooling.

8. Implementation Details

The complete program is about 150 lines. The most important parts are described below.

8.1 Data storage and averaging

```
students = {}

def calculate_average(marks):
    if not marks:
        return 0.0
    return sum(marks) / len(marks)
```

8.2 Adding a student with validation

`add_student()` reads the name, rejects empty and duplicate names, reads the marks line, rejects an empty line, and converts every token with a list comprehension. A `ValueError` from `int()` is caught and reported. Only when every check passes is the marks list stored.

```
try:
    marks = [int(mark) for mark in marks_input.split()]
except ValueError:
    print("Marks must be numbers separated by spaces.\n")
    return

students[name] = marks
```

8.3 Finding the topper

The function scans all students, keeping the best average seen so far. It starts from a sentinel of -1 and updates only on a strictly greater average. `find_lowest_scorer()` mirrors this logic but starts from `NONE` and uses a strictly smaller comparison.

```
for name, marks in students.items():
    average = calculate_average(marks)
    if average > topper_average:
        topper_average = average
        topper_name = name
```

8.4 The main loop

`main()` runs an infinite `while True` loop that shows the menu, reads the choice, and dispatches with an `if/elif` chain. Choice 7 prints a farewell and leaves the loop with `break`; anything else prints an invalid-choice message.

```
while True:
    show_menu()
    choice = input("Enter your choice (1-7): ").strip()
    if choice == "1":
        add_student()
    ...
    elif choice == "7":
        print("Goodbye!")
        break
    else:
        print("Invalid choice. Please enter a number from 1 to 7.\n")
```

9. Results

The program was executed with scripted input to capture the real output below. Three students were added, then the full list, topper and lowest scorer were displayed, Bob's marks were updated, Alice was removed, and the list was shown again. The menu is displayed before every prompt but is printed once here to save space.

```
===== STUDENT GRADE MANAGER =====
1. Add student
2. View all students
3. Find topper
4. Find lowest scorer
5. Update a student's marks
6. Remove a student
7. Exit
Enter your choice (1-7): 1
Enter student name: Alice
Enter marks separated by spaces: 85 90 78
Added Alice with marks [85, 90, 78].

Enter your choice (1-7): 1
Enter student name: Bob
Enter marks separated by spaces: 70 65 80
Added Bob with marks [70, 65, 80].

Enter your choice (1-7): 1
Enter student name: Charlie
Enter marks separated by spaces: 92 88 95
Added Charlie with marks [92, 88, 95].

Enter your choice (1-7): 2

--- All Students ---
Alice: marks=[85, 90, 78], average=84.33
Bob: marks=[70, 65, 80], average=71.67
Charlie: marks=[92, 88, 95], average=91.67

Enter your choice (1-7): 3
Topper: Charlie with average 91.67

Enter your choice (1-7): 4
Lowest scorer: Bob with average 71.67

Enter your choice (1-7): 5
Enter student name to update: Bob
Enter new marks separated by spaces: 75 80 85
Updated Bob's marks to [75, 80, 85].

Enter your choice (1-7): 6
Enter student name to remove: Alice
Removed Alice.

Enter your choice (1-7): 2

--- All Students ---
Bob: marks=[75, 80, 85], average=80.00
Charlie: marks=[92, 88, 95], average=91.67

Enter your choice (1-7): 7
Goodbye!
```

The averages are correct: $(85+90+78)/3 = 84.33$, $(70+65+80)/3 = 71.67$ and $(92+88+95)/3 = 91.67$, so Charlie is the topper and Bob is the lowest scorer. After the update Bob's average becomes $(75+80+85)/3 = 80.00$. A second session exercised the validation paths:

```

===== STUDENT GRADE MANAGER =====
1. Add student
2. View all students
3. Find topper
4. Find lowest scorer
5. Update a student's marks
6. Remove a student
7. Exit
Enter your choice (1-7): 3
No students added yet.

Enter your choice (1-7): 1
Enter student name:
Student name cannot be empty.

Enter your choice (1-7): 1
Enter student name: Dave
Enter marks separated by spaces:
Please enter at least one mark.

Enter your choice (1-7): 1
Enter student name: Dave
Enter marks separated by spaces: abc
Marks must be numbers separated by spaces.

Enter your choice (1-7): 1
Enter student name: Eve
Enter marks separated by spaces: 60 70
Added Eve with marks [60, 70].

Enter your choice (1-7): 1
Enter student name: Eve
Eve already exists. Please use the update option.

Enter your choice (1-7): 50
Invalid choice. Please enter a number from 1 to 7.

Enter your choice (1-7): 5
Enter student name to update: Zed
Zed not found.

Enter your choice (1-7): 6
Enter student name to remove: Zed
Zed not found.

Enter your choice (1-7): 9
Invalid choice. Please enter a number from 1 to 7.

Enter your choice (1-7): 7
Goodbye!

```

10. Testing Approach

Testing was black-box and manual-style but automated by script: each case feeds a fixed sequence of inputs to the program, captures the output, and compares the last message with the expected text. Cases cover normal use, every validation branch, empty-state behaviour, and edge cases. All 16 cases produced the expected output. The two cases marked *Limitation* confirm behaviour that works as coded but is not ideal (see Challenges and Future Enhancements).

#	Test case	Input sequence	Expected result	Result
1	Add a valid student	1 > Alice > 85 90 78	Added Alice with marks [85, 90, 78].	Pass
2	Empty student name	1 > (blank)	Student name cannot be empty.	Pass

#	Test case	Input sequence	Expected result	Result
3	Duplicate student name	1 > A > 50 > 1 > A	A already exists. Please use the update option.	Pass
4	Non-numeric marks	1 > Dave > abc	Marks must be numbers separated by spaces.	Pass
5	Empty marks input	1 > Dave > (blank)	Please enter at least one mark.	Pass
6	View with no students	2	No students added yet.	Pass
7	Find topper	1 > A > 50 60 > 1 > B > 90 80 > 3	Topper: B with average 85.00	Pass
8	Find lowest scorer	1 > A > 50 60 > 1 > B > 90 80 > 4	Lowest scorer: A with average 55.00	Pass
9	Update existing student	1 > A > 50 > 5 > A > 70 80	Updated A's marks to [70, 80].	Pass
10	Update missing student	5 > Zed	Zed not found.	Pass
11	Remove existing student	1 > A > 50 > 6 > A	Removed A.	Pass
12	Remove missing student	6 > Zed	Zed not found.	Pass
13	Invalid menu choice	9	Invalid choice. Please enter a number from 1 to 7.	Pass
14	Tie for topper (equal averages)	1 > A > 80 > 1 > B > 80 > 3	Topper: A with average 80.00	Pass
15	Out-of-range marks (-5, 200)	1 > N > -5 200	Added N with marks [-5, 200].	Limitation
16	Topper when all averages are below -1	1 > N > -5 > 3	Topper: None with average -1.00	Limitation

Key test ideas were: boundary inputs (empty strings, single mark), invalid types (letters instead of numbers), state-dependent behaviour (operations on an empty list), duplicate and missing keys, and ties between averages.

11. Challenges Faced

- **Input validation:** deciding which mistakes to catch and making sure every one returns to the menu with a helpful message rather than a traceback.
- **Empty-state handling:** view, topper and lowest-scorer must not run on an empty dictionary; each starts with a guard.
- **Choosing initial values for comparisons:** the topper search uses a sentinel of -1, which works for normal marks but reports "None" if every average is below -1 (possible only because negative marks are accepted). The lowest-scorer search avoids this by starting with None.
- **Ties:** the strict comparisons report only the first student when averages are equal, which had to be understood and documented.

- **Update vs. add:** keeping the two flows distinct (duplicate names are rejected on add, missing names on update) required careful checks on the dictionary keys.
- **Designing diagrams for a procedural program:** without classes, the class diagram had to be re-expressed as a component diagram.

12. Learnings & Key Takeaways

- Dictionaries and lists are powerful enough to model a small database when the key and value are chosen well.
- Breaking a program into single-purpose functions makes it easier to test, explain and extend.
- Defensive programming (strip, empty checks, try/except) is what separates a demo from a dependable tool.
- Edge cases such as ties, empty data and sentinel values are where subtle bugs hide, so they deserve explicit tests.
- UML diagrams (use case, flowchart, sequence, component, ER) help communicate a design before and after coding.

13. Future Enhancements

- **Persistence:** save and load data with JSON or CSV, or move to SQLite so records survive between runs (the ER model in Figure 5 maps directly to two tables).
- **Range validation:** accept only marks within 0-100 (or a configurable maximum) and fix the topper sentinel.
- **Subject-wise marks and grades:** store marks per subject, add letter grades, pass/fail status and class average.
- **Ranking and ties:** show a sorted leaderboard and report every student tied for first or last place.
- **Usability:** case-insensitive names, confirmation before removing a student, and an option to export a report.
- **Quality:** add automated unit tests (for example with pytest), guard the entry point with `if __name__ == "__main__":`, and consider a class-based design or a simple GUI/web front end.

14. References

- [1] Python Software Foundation, *The Python 3 Documentation and Python Tutorial*. <https://docs.python.org/3/>
- [2] Python Software Foundation, *Built-in Functions: input(), int(), sum(), len() and Built-in Types: dict, list, str*. <https://docs.python.org/3/library/>
- [3] G. van Rossum, B. Warsaw, N. Coghlan, *PEP 8 – Style Guide for Python Code*. <https://peps.python.org/pep-0008/>
- [4] Object Management Group, *Unified Modeling Language (UML) Specification*. <https://www.omg.org/spec/UML/>

[5] I. Sommerville, *Software Engineering*, Pearson.

[6] ReportLab Inc., *ReportLab PDF Library User Guide* (used to generate this report).
<https://www.reportlab.com/docs/reportlab-userguide.pdf>